University of Liberal Arts Bangladesh

Department of Computer Science & Engineering

CSE 2201 : Algorithms

Lecture 11: Quick Sort

Atanu Shuvam Roy

Lecturer,
Dept. of CSE, ULAB, Dhaka

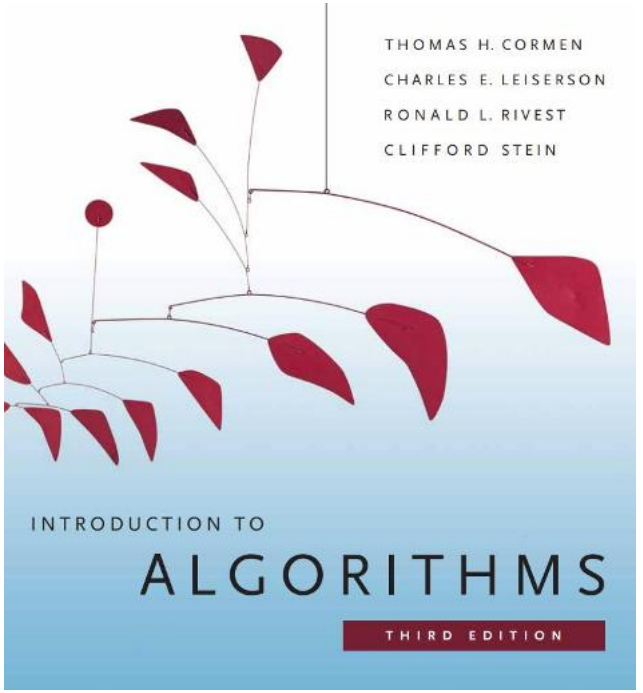
EMAIL:
atanu.shuvam@ulab.edu.bd

Class Code
GVWTHYJV

Fall 2025

Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100

Sorting problem

Ex. Student records in a university.

item →

key →

Chen	3	A	991-878-4944	308 Blair
Rohde	2	A	232-343-5555	343 Forbes
Gazsi	4	B	766-093-9873	101 Brown
Furia	1	A	766-093-9873	101 Brown
Kanaga	3	B	898-122-9643	22 Brown
Andrews	3	A	664-480-0023	097 Little
Battle	4	C	874-088-1212	121 Whitman

Sort. Rearrange array of N items into ascending order.

Andrews	3	A	664-480-0023	097 Little
Battle	4	C	874-088-1212	121 Whitman
Chen	3	A	991-878-4944	308 Blair
Furia	1	A	766-093-9873	101 Brown
Gazsi	4	B	766-093-9873	101 Brown
Kanaga	3	B	898-122-9643	22 Brown
Rohde	2	A	232-343-5555	343 Forbes

Today's Contents

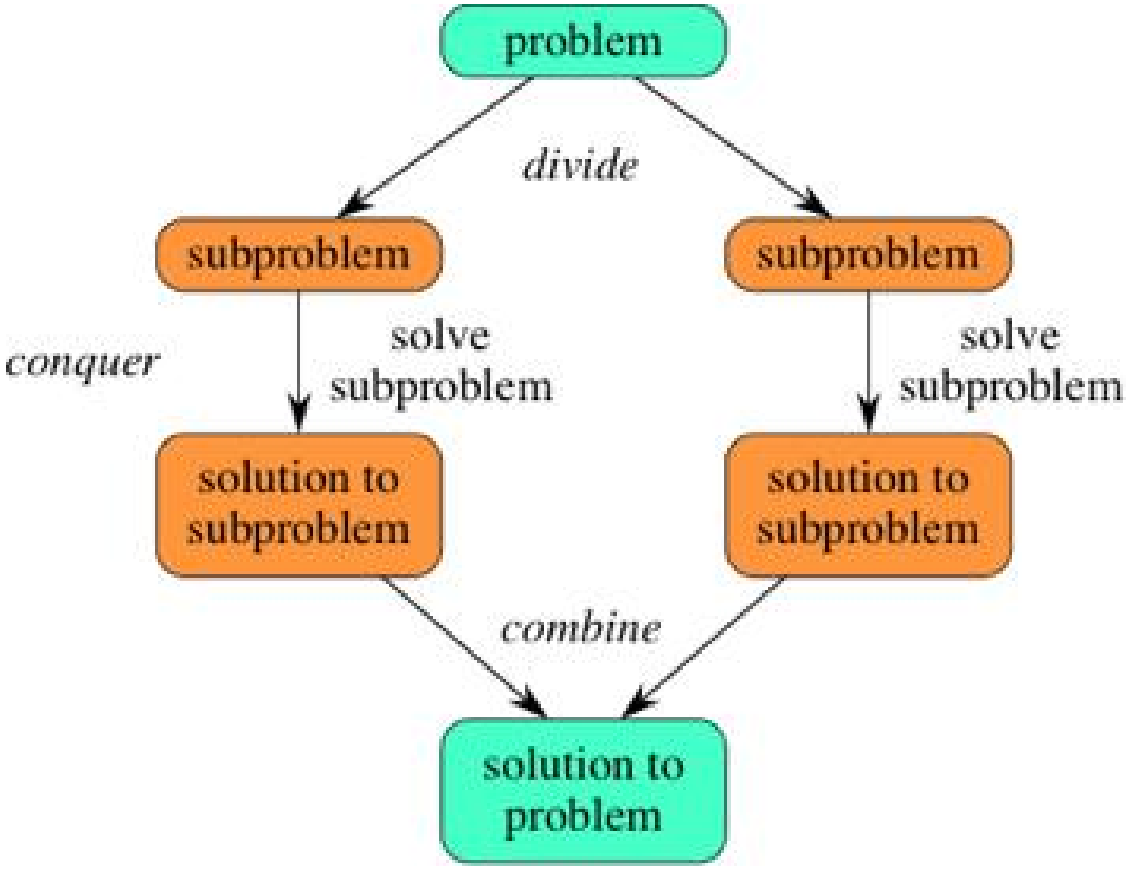
- Quick Sort
- Complexity Analysis
- Questions for You!

Divide & Conquer Approach

The **divide-and-conquer** paradigm involves **three steps** at each level of the recursion:

1. **Divide** the problem into a number of **subproblems**.
2. **Conquer** the subproblems by solving them recursively. If the subproblem sizes are small enough, however, just solve the subproblems in a straightforward manner.
3. **Combine** the solutions to the subproblems into the solution for the original problem.

Divide & Conquer Approach



Quick Sort

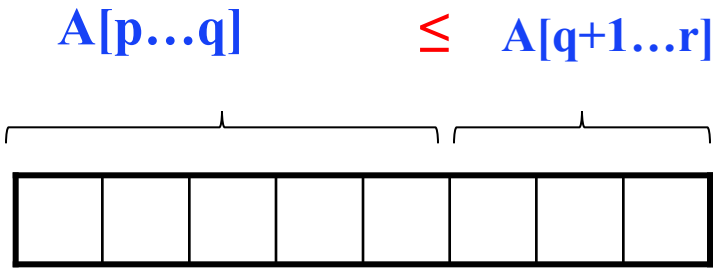
- The **Quick sort** algorithm closely follows the **divide-and-conquer** paradigm.

1. **Divide**:
    - Partition the array A into 2 subarrays A[p..q-1] and A[q+1..r], such that each element of A[p..q-1] is smaller than or equal to each element in A[q+1..r]
2. **Conquer**:
    - Recursively sort A[p..q] and A[q+1..r] using Quicksort
3. **Combine**:
    - Trivial: the arrays are sorted in place
    - No additional work is required to combine them
    - The entire array is now sorted

Quick Sort

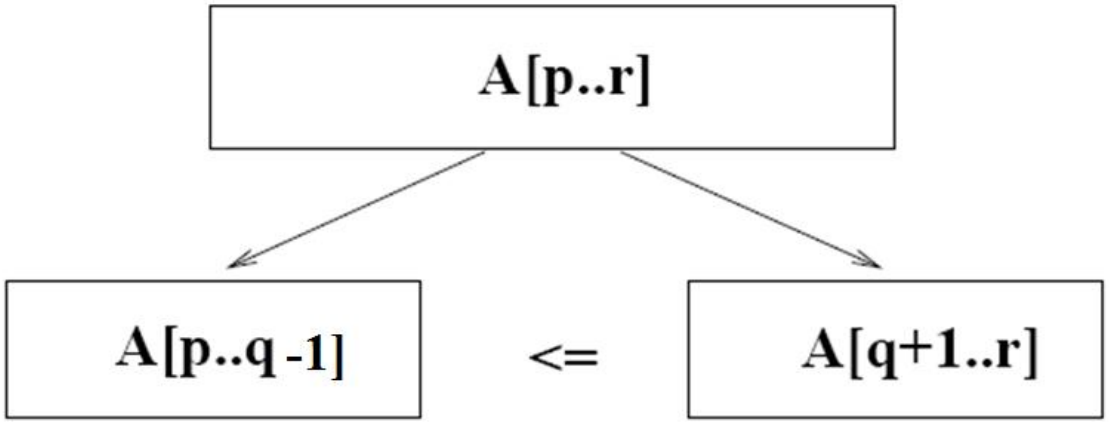
Divide

- Partition the array A into 2 subarrays **A[p..q-1]** and **A[q+1..r],** such that each element of **A[p..q-1]** **is smaller than or equal to each element in A[q+1..r]**



Quick Sort

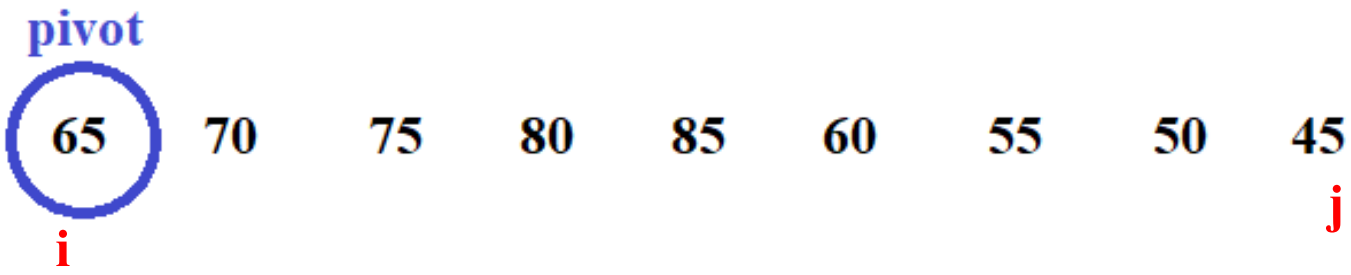
Need to find index **q (pivot)** to **partition** the array



Array to Sort

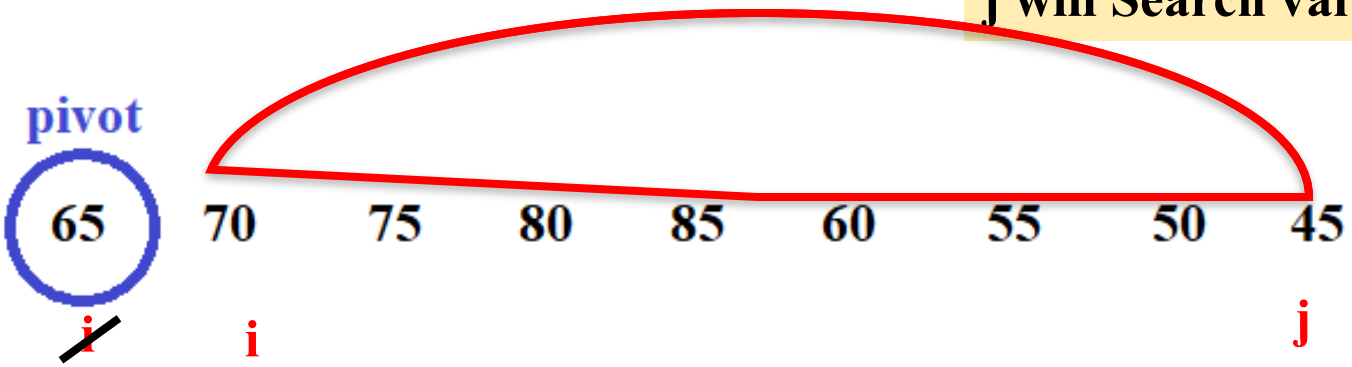
65	70	75	80	85	60	55	50	45

Select a Pivot to compare

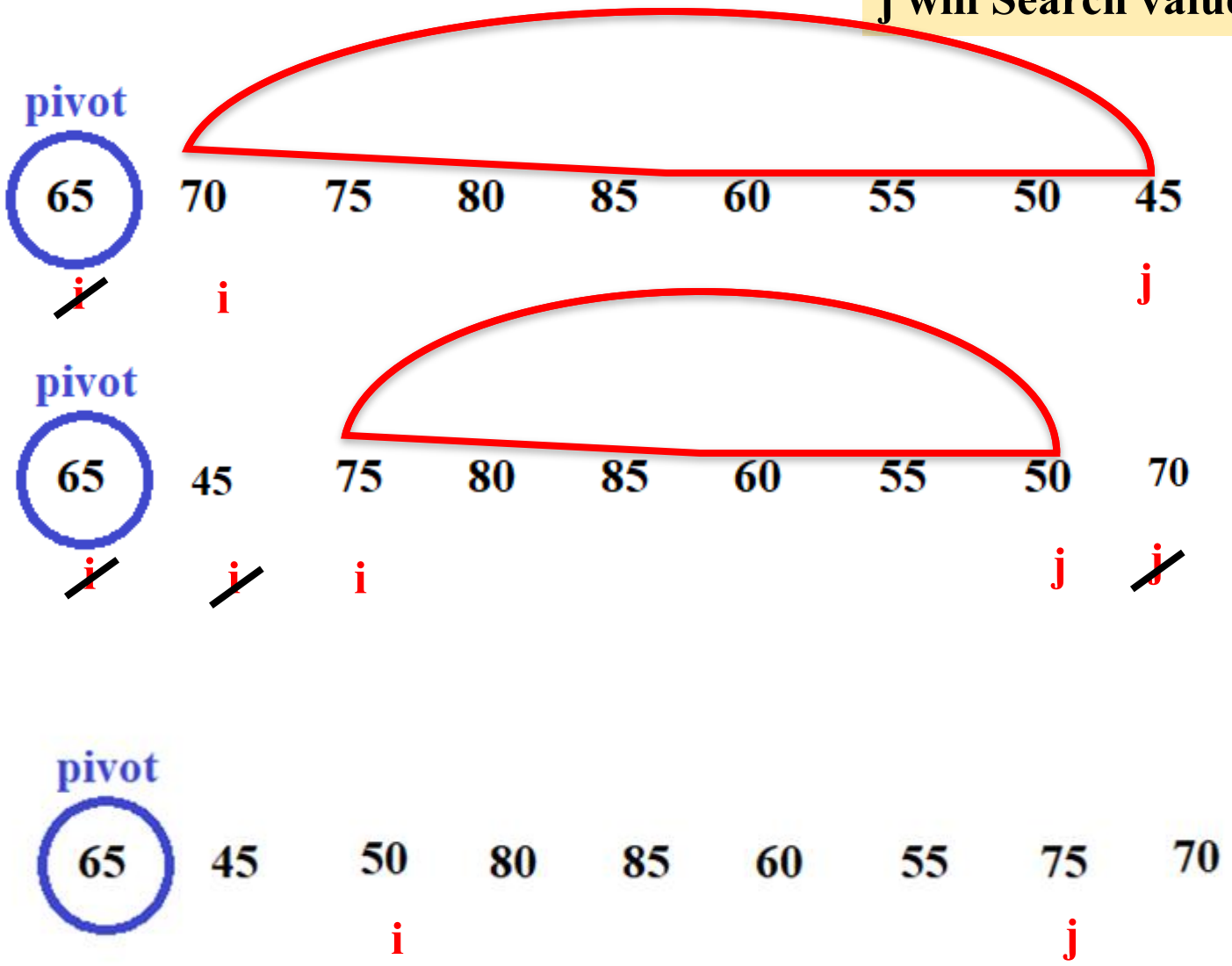


i ← left
j ← right

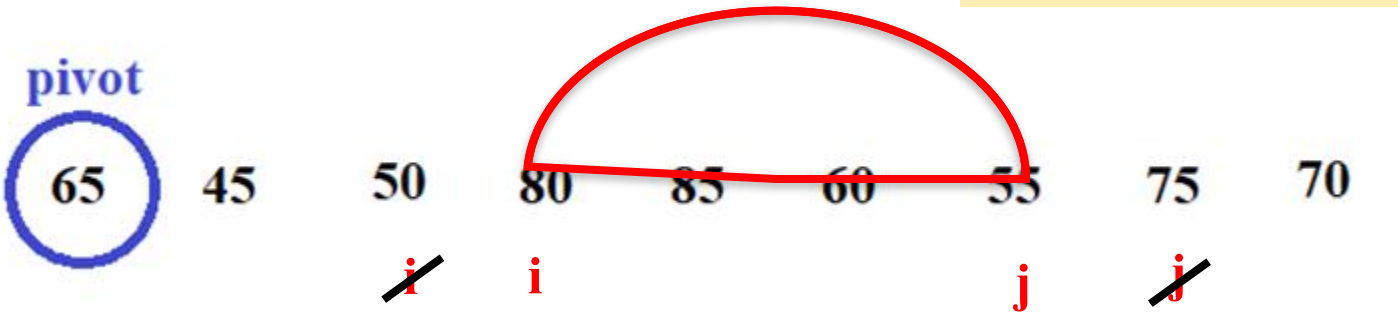
i will Search value > pivot
j will Search value <= pivot

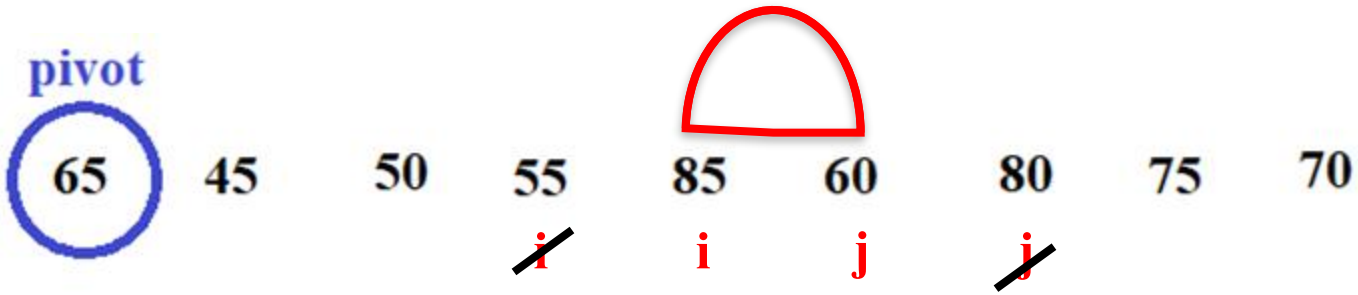


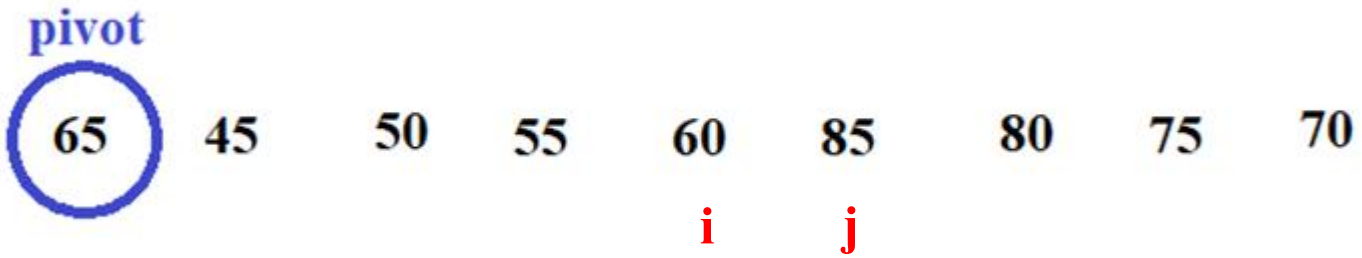
i will Search value > pivot
j will Search value <= pivot



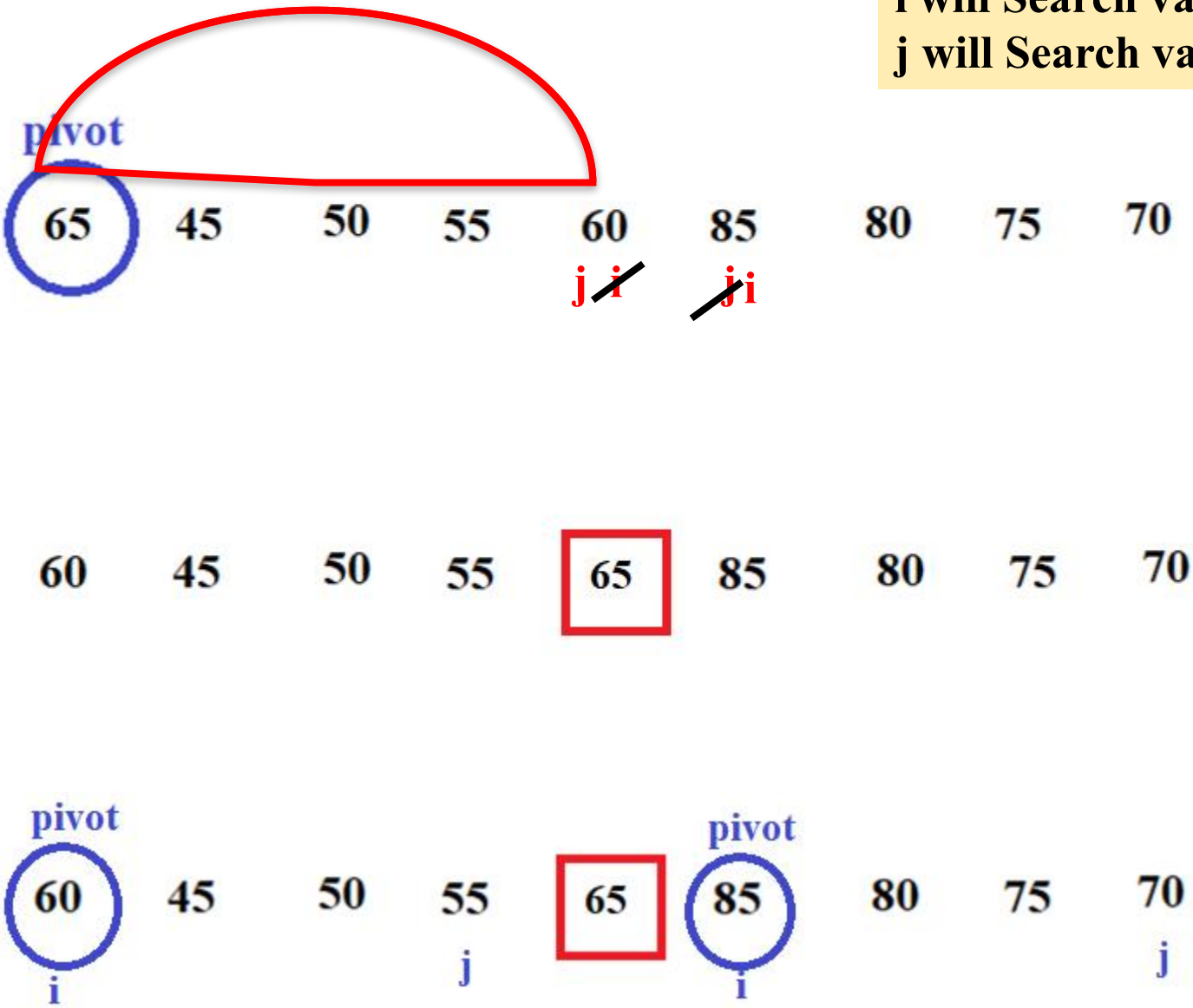
i will Search value > pivot
j will Search value <= pivot





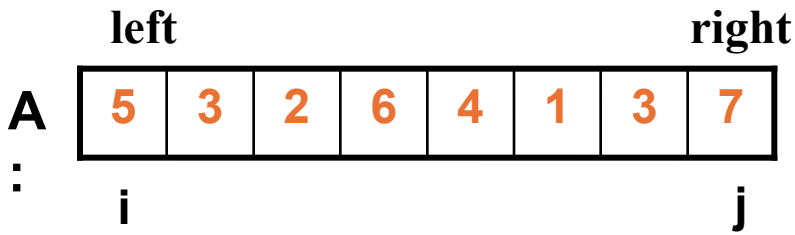


i will Search value > pivot
j will Search value <= pivot



Algorithm of Quick Sort

```
QUICKSORT (A, left, right)
1.    if (left < right)
2.        q = PARTITION(A, left, right)
3.        QUICKSORT (A, left, q-1)
4.        QUICKSORT (A, q+1, right)
```




```
PARTITION (A, left, right)
1.    pivot ← A[left]
2.    i ← left
3.    j ← right
4.    while (i < j)
5.        while ( A[i] < = pivot && i <= right)
6.            i = i+1
7.        while ( A[j] > pivot )
8.            j = j – 1
9.        if ( i < j )
10.           A[i] ↔ A[j]
11.   A[j] ↔ pivot
12.   return j
```

Time Complexity

- **Best Case Partitioning**
  - **Partitioning (q) produces two regions of size n/2**
- **Worst Case Partitioning**
  - **One region has one element and the other has n – 2 elements**

Time Complexity

- **Best-case partitioning**
  - **Partitioning (q) produces two regions of size n/2**

```
QUICKSORT (A, left, right)
    if (left < right)
        q = PARTITION(A, left, right)        → O(n)
        QUICKSORT (A, left, q-1)             → T(n/2)
        QUICKSORT (A, q+1, right)            → T(n/2)
```

Complexity:

$$T(n) = O(n) * [T(n/2) + T(n/2)]$$
$$= O(n) * 2T(n/2)$$
$$= O(n) * O(\lg n)$$
$$= O(n \lg n)$$

Time Complexity

- **Worst-case partitioning**
  - **One region has 1 element and the other has n – 1 elements**

```
QUICKSORT (A, left, right)
    if (left < right)
        q = PARTITION(A, left, right)        → O(n)
        QUICKSORT (A, left, q-1)             → T(1)
        QUICKSORT (A, q+1, right)            → T(n-2)
```

Complexity:

$$T(n) = O(n) * [T(1) + T(n-2)]$$
$$= O(n) * [1 + O(n)]$$
$$= O(n) * O(n)$$
$$= O(n^2)$$

Complexity Analysis of Three Algorithms

Algorithm	Best case	Worst case
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$
Merge Sort	$O(nlogn)$	$O(nlogn)$
Quick Sort	$O(nlogn)$	$O(n^2)$

Advantage and Drawbacks

- **Advantages**
  - The quick sort is regarded as the best sorting algorithm.
  - This is because of its significant advantage in terms of efficiency because it is able to deal well with a huge list of items.
  - It sorts in place, no additional storage is required as well.

- **Disadvantage**
  - The slight disadvantage of quick sort is that its worst-case performance is similar to average performances of the bubble, insertion or selections sorts.

In general, the quick sort produces the most effective and widely used method of sorting a list of any item size.

Question for you

1. Which Algorithm will you choose?
   - For Sorted Data List
   - For Unsorted Data List

2. Is Quick sort a In-Place Algorithm?

3. Sort the given data set using both the Quick Sort. Show each steps.

15	8	13	18	15	14

Some References

https://visualgo.net/en/sorting

Reference

- http://coursera.org/learn/algorithms-part1/
- GeeksForGeeks
- YouTube
- LLMs
- Old Lectures of ULAB (Pramanik Sir)

Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh

Department of Computer Science & Engineering